# Eagle Delivery: Search and Q-Learning for a Campus Delivery Robot

COSC 461/561 Introduction to Artificial Intelligence · Project 1 · Moe Shrime · Eastern Michigan University · October 2026

## Part A: Planning a delivery with A*

### (a) Problem formulation

- **State:** the robot's cell as a `(row, column)` tuple. Any non-# cell, including building letters, is a valid state.
- **Initial state:** the start building's entrance, for example S = (1, 1). **Actions:** UP, DOWN, LEFT, RIGHT (no diagonals).
- **Transition model:** deterministic. Result(s, a) is the neighboring cell in direction a if that cell is not blocked. Search only generates moves that change the position (`campus.neighbors`).
- **Goal test:** the state equals the destination entrance. **Path cost:** each move costs 1, so path cost = number of moves.

**Why BFS is optimal here.** BFS uses a FIFO queue, so it removes nodes in order of depth: all depth-1 nodes, then all depth-2 nodes, and so on. Every step costs 1, so a node's depth equals its path cost. The first time BFS removes the goal, no cheaper route can exist, because every cheaper route would be shallower and would have been reached first.

### (b) Manhattan heuristic

`manhattan(s, g)` returns $h(s) = |r_s - r_g| + |c_s - c_g|$. Each move changes either the row or the column by exactly 1. So any route needs at least $|\Delta r|$ vertical moves and $|\Delta c|$ horizontal moves, and walls can only force extra moves. This means $h(s) \le h^*(s)$, the true cost to the goal, for every state, so h is **admissible**. It also gives h(goal) = 0.

### (c) A* implementation and results

`astar` keeps a heap of `(f, insertion number, g, state)` with f = g + h, plus dictionaries for the best known g-cost and parent of each state. Successors come from `campus.neighbors` in UP, DOWN, LEFT, RIGHT order. A successor is added (or re-added) only if its new g is strictly cheaper. When a popped entry's g no longer matches the best g, it is skipped as stale. The goal test happens when a non-stale entry is removed, and a node counts as expanded when its successors are generated. All 14 public tests pass, and BFS reproduces the expected S→W result (18 moves, 67 expansions).

| Map | Delivery task | BFS route length | A* route length | BFS expanded | A* expanded | A* saving |
|---|---|---|---|---|---|---|
| Baseline | S Student Center → W Walton Hall | 18 | 18 | 67 | 22 | 67% |
| Baseline | L Halle Library → D Downing Hall | 22 | 22 | 79 | 62 | 22% |
| Baseline | H Pray-Harrold → P Putnam Hall | 16 | 16 | 41 | 16 | 61% |
| Detour | S Student Center → W Walton Hall | 26 | 26 | 70 | 51 | 27% |
| Detour | L Halle Library → D Downing Hall | 22 | 22 | 65 | 60 | 8% |
| Detour | H Pray-Harrold → P Putnam Hall | 16 | 16 | 41 | 16 | 61% |

**Route lengths are equal in all six tasks.** Both algorithms are optimal here: BFS because every step costs 1, and A* because the Manhattan heuristic is admissible. **A* expanded fewer nodes in all six tasks, but the savings vary a lot (8% to 67%).** The savings depend on how close h is to the true cost. For H→P, h(H) = 16 equals the true cost, so A* expands only the 16 nodes on its route. For L→D, the best route must climb up into row 3 twice. Those UP moves raise h, so the true cost is far above the estimate, many nodes tie on f, and A* explores almost as widely as BFS. On the detour map, the closure at (3, 6) turns the 18-move S→W route into a 26-move route while h(S) stays 14. A* first follows the now-blocked direct direction, which is why its saving drops from 67% to 27%. So A* does not always expand far fewer nodes. When walls make the true cost much larger than h, it behaves almost like BFS, and with unlucky ties it could even expand more.

## Part B: Learning a delivery policy with Q-learning

**Implementation.** `epsilon_greedy` draws `rng.random()`. With probability ε it returns `rng.choice` over all four actions. Otherwise it returns an action with the largest Q(s, a), where unseen pairs count as 0.0 and ties are broken with `rng.choice`. `q_learning_update` uses the target y = r for a terminal transition and $y = r + \gamma \cdot \max_{a'} Q(s', a')$ otherwise, with unseen actions included at 0. It then applies $Q(s,a) \leftarrow Q(s,a) + \alpha[y - Q(s,a)]$, which is the lecture's running average $(1-\alpha)Q + \alpha \cdot$sample. Settings: 2,000 training episodes, seeds 0/1/2, α = 0.3, γ = 0.95, ε = 0.2, slip = 0, 200-action cap, and 100 greedy evaluation episodes (ε = 0, learning frozen). All values below are means over the 3 seeds, from `results/training_stats.csv`.

| Task | BFS optimum | Timed-out training episodes | First optimal training episode | Training steps, ep. 1–100 | Training steps, last 500 | Share of last 500 that were optimal | Eval success | Eval steps / return |
|---|---|---|---|---|---|---|---|---|
| S → W | 18 | 17.0 | 103 | 98.0 | 22.5 | 5.7% | 100% | 18 / 22 |
| L → D | 22 | 8.7 | 140 | 93.9 | 27.7 | 2.8% | 100% | 22 / 18 |
| H → P | 16 | 15.3 | 62 | 57.6 | 20.1 | 7.5% | 100% | 16 / 24 |

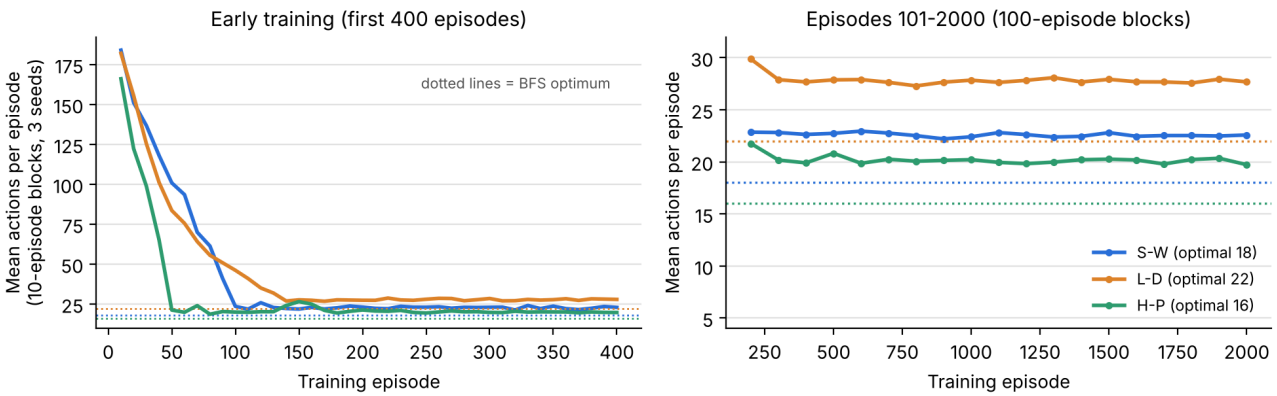



**Figure 1.** Mean actions per training episode, averaged over seeds 0–2. Left: first 400 episodes in 10-episode blocks. Right: the rest of the run in 100-episode blocks. Dotted lines mark the BFS optimum for each task.

- **Final performance is optimal and fully consistent.** All 9 training runs reach a 100% evaluation success rate, and every greedy delivery uses exactly the BFS-optimal number of actions (18, 22, 16). The returns are 40 − L = 22, 18 and 24, with a standard deviation of 0 across seeds. Q-learning found the shortest routes from experience alone, without seeing the map.
- **Learning is fast and happens early.** The first episodes hit the 200-action limit because the table is empty and the robot wanders. The goal reward then spreads backward one state per update. In 8 of the 9 runs, the last timeout came between episodes 12 and 41, and every episode after that was a successful delivery. The only exception, H→P with seed 0, had one more timeout at episode 149. After episode 100, the curves in Figure 1 are flat.
- **Training behavior stays above the optimum, while the learned policy does not.** With ε = 0.2, a random action is taken 20% of the time, and 3 out of 4 random actions are not the best move. So each step goes wrong with probability about 0.15, and a full route has no wrong steps with probability $0.85^{L}$. That predicts 5.4%, 2.8% and 7.4% optimal episodes, which closely matches the measured 5.7%, 2.8% and 7.5%. L→D is the longest route, so it shows the largest gap (27.7 vs 22) and was the last to produce an optimal training episode (140).

## Part C: My campus map and an exploration experiment

```
####################
#S........#########W#
#####.#############.#
#####....L#########.#
#####.#############.#
#####...............#
#####.#########.###.#
#####.#########.###D#
#####.#########.#####
#.................#####
#.#############.#####
#H#############P#####
####################
```

**(a) Map design.** `maps/my_campus.txt` is 13 × 21, rectangular, surrounded by #, and uses each of S, L, H, D, P and W once. It has 62 walkable cells.

- **Loop:** the row-5 and row-9 corridors joined by columns 5 and 15 form one cycle. From (5, 5), the robot can take the top of the loop to W (**26 moves total**) or go down and around the bottom (34 moves, 8 extra).
- **Dead ends:** the top corridor (1, 6)–(1, 9) points straight at W but is walled off. It is a trap for anything that just heads toward W. The L spur (3, 6)–(3, 9) and the D, H and P spurs are also dead ends.
- **Search on S→W:** BFS finds 26 moves with **61** expansions. A* finds 26 moves with **34** expansions.

A* walks into the top dead end and the L spur first. Each step there lowers h by 1 and raises g by 1, so f stays at 19 and 23, the lowest values on the frontier. A* never enters the bottom of the loop, though, because f = 28 there is above the optimal cost of 26. BFS expands 61 of the 62 cells.

### (b, c-i) Question and prediction

> **Question:** How does the exploration rate ε affect learning speed, training performance, and final performance on my map's S→W task?
> **Prediction (made before running):** (1) The final greedy policy will be optimal (26 moves) for every ε, because the map is small and 2,000 episodes is plenty. (2) Training cost will grow with ε: each step goes wrong with probability about 0.75ε, so the share of fully optimal training episodes should be about $(1-0.75\varepsilon)^{26}$, which is 37%, 1.5% and about 0%. (3) ε = 0.05 might discover the route most slowly, because it explores the least.

### (c-iii) Method

Only ε changed, set to 0.05, 0.2 and 0.5. Everything else was fixed: `my_campus.txt`, S→W, α = 0.3, γ = 0.95, slip = 0, 2,000 training episodes, a 200-action cap, and seeds 0, 1 and 2. Each seed started from a fresh, empty Q-table, for 9 runs in total.

After training, each table was frozen and evaluated greedily (ε = 0) for 100 episodes. Commands: `run_project.py train --map maps/my_campus.txt --epsilon E --output results/custom_eXXX`, then `summarize_results.py`.

### (c-iv) Results

| ε | Timed-out training episodes | First 20-success streak starts | First optimal (26-step) episode | Training steps, last 500 | Training return, last 500 | Optimal share, last 500 | Total training actions | Eval success / steps / return |
|---|---|---|---|---|---|---|---|---|
| 0.05 | 22.3 | ep. 37 | ep. 86 | 27.4 | 12.6 | 37.9% | 62,507 | 100% / 26 / 14 |
| 0.2 | 23.7 | ep. 54 | ep. 106 | 32.8 | 7.2 | 1.6% | 73,231 | 100% / 26 / 14 |
| 0.5 | 31.3 | ep. 60 | never | 53.9 | −13.9 | 0.0% | 115,298 | 100% / 26 / 14 |

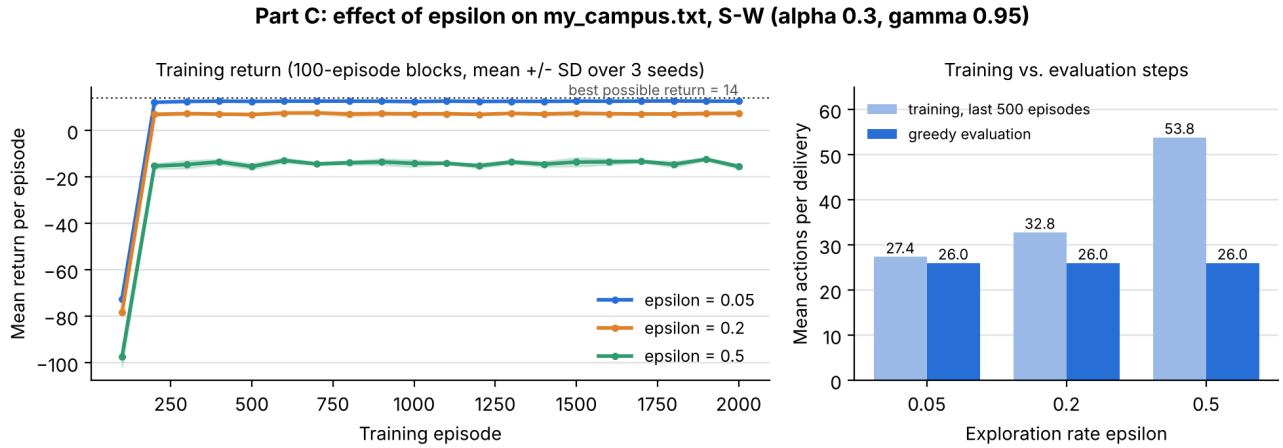



**Figure 2.** Left: training return per 100-episode block (mean ± SD over 3 seeds). Right: mean actions per delivery in the last 500 training episodes compared with greedy evaluation.

### (c-v) Discussion

- **Predictions 1 and 2 held.** Every one of the 9 runs evaluates at 100% success in exactly 26 moves (return 14). The agent always takes the top of the loop and never the 34-move bottom path or a dead end. So ε made **no difference to final performance**. Training cost, however, rose steeply: about 1.4, 6.8 and 27.9 extra moves per delivery over the optimum. ε = 0.5 used almost twice as many total actions as ε = 0.05. The optimal-episode shares (37.9%, 1.6%, 0%) are very close to the predicted 37%, 1.5% and about 0%. With ε = 0.5, not a single training episode was optimal in 6,000 tries.
- **Prediction 3 was wrong.** ε = 0.05 learned *fastest*: its first optimal episode came at 86 (vs. 106 and never), and its 20-success streak started first (37 vs. 54 and 60). The reason is that unseen Q-values start at 0, while every action already tried has a negative value (−1 per step). So the greedy choice already prefers untried actions, much like the exploration functions from lecture, and the robot explores even when ε is small. Extra random moves mostly undo progress that greedy actions just made.
- **Limits.** This map is small, deterministic and mostly corridors, so all settings converged by about episode 100. Whether a low ε would get stuck on the longer loop route on a bigger map, or with slip, was not tested here. The main lesson is the difference between learning and acting: ε changes how well the robot behaves *while* it learns, not the quality of the policy it learns here.

**Reproducibility.** All raw CSVs (`training_seed*.csv`, `evaluation_seed*.csv`, `summary.csv`, `learning_curve.csv`) are in `results/`. `search_table.py` rebuilds the search tables, and `analyze_results.py` rebuilds `training_stats.csv` and both figures.